CS6650 Assignment 3 — Performance Report

March 27, 2026
Name: Suiyang Mai

Screenshots and the degradation plot are in **`HW3/load-tests/results/`** (the five batch runs are under **`batch-tuning-screenshots/`**). Filenames are in the table below.

Topic	File (under `load-tests/results/`)
Metrics API — client log / terminal	`fig-metrics-api-curl.png`
RabbitMQ queues	`fig-rabbitmq-queue-rates.png`
Activity Monitor CPU (MAIN)	`fig-activity-monitor-cpu-main.png`
Activity Monitor memory (MAIN)	`fig-activity-monitor-memory-main.png`
Activity Monitor memory (consumer JVM)	`fig-activity-monitor-memory-consumer-main.png`
Batch tuning (five runs)	`batch-tuning-screenshots/hw3-batch-exp01-100x100-main.png` … `exp05-5000x1000-main.png`
Degradation curve (script-generated)	`fig-degradation-latency-p95.png`

---

1. Environment

Component	Setting
Server	`server-v3`, WebSocket producer, `GET /api/metrics` on **8081**
Consumer	`consumer-v3`, batched JDBC, RabbitMQ manual ack, DLQ `db.write.dlq`
Broker	RabbitMQ Management UI `http://localhost:15672`
Database	PostgreSQL 16 (Homebrew), `cs6650_chat`, user `maisuiyang`
Load client	`HW1/client-part2`: WARMUP 32k → BASELINE 500k → MAIN 1M, 32 workers (26 in Endurance)
RabbitMQ listeners (consumer)	`concurrency` **20**, `max-concurrency` **20**, `prefetch` **50**, manual ack (`consumer-v3/.../application.properties`)
DB writer / stats (consumer)	One **`database-writer`** thread for batched JDBC; **`statistics-aggregator`** thread for `analytics_minute` refresh

Component	Setting
Resilience4j (consumer)	`roomMessages` / `databaseWrites` circuit breakers (see `application.properties` for window size, failure rate, open-state wait)
Resilience4j (server)	`metricsApi` rate limiter **60** calls / **1 s** (`server-v3/.../application.properties`)

Final consumer persistence: `chat.persistence.batch-size=5000`, `chat.persistence.flush-interval-ms=1000` (`consumer-v3/src/main/resources/application.properties`).

---

2. Write performance

2.1 Throughput (client aggregate)

The client writes `*-throughput.csv` bucket rows. For **MAIN (1M messages)**, the bucket timestamps in `MAIN-throughput.csv` span **120 s** with **1,000,000** messages counted in the buckets → **≈ 8,333 msg/s** aggregate send-side over that window. **BASELINE (500k)** sums to **500,000** messages in `BASELINE-throughput.csv`; bucket spacing is **10 s**, so the file is coarse but matches a short, high-rate phase.

These numbers are **send-side** (generator + WebSocket workers), not PostgreSQL commits per second. The consumer batches inserts, so `xact_commit` stays small compared to message volume.

MAIN ran at about **8.3k msg/s** for **120 s** (1M messages); BASELINE was **500k** with **10 s** buckets in the CSV, so the throughput view is rougher. End-to-end ACK **p99** was actually **lower on MAIN than BASELINE** here (§2.2), which likely reflects phase timing and batching rather than a simple "more load = always worse latency" pattern.

p95 ACK latency vs. progress through each phase was plotted from `BASELINE-latency.csv` and `MAIN-latency.csv`. Output: **`load-tests/results/fig-degradation-latency-p95`**

2.2 Latency (client ACK, ms)

From `*-latency.csv` (`latency` column), with the client sampling every 10th message (`LATENCY_SAMPLE_EVERY_N = 10`).

Phase	Samples	p50	p95	p99	Min	Max
BASELINE	42,104	11,188	14,710	15,022	0	15,195
MAIN	2,398	2,216	5,872	6,260	0	7,561

BASELINE has higher median and tail latency than MAIN in this dataset. Both are end-to-end ACK latency (WebSocket path + server), not JDBC-only.

Room history by time is **database** latency, not the WebSocket load-test path. `EXPLAIN (ANALYZE, BUFFERS)` on `messages` with `room_id = '1'`, `sent_at` in the last day, `ORDER BY sent_at ASC`, `LIMIT 1000`: **Planning Time** 3.43 ms, **Execution Time 18.15 ms** (1000 rows). Plan: **`idx_messages_sent`**

+ `room_id` **filter** (18,736 rows removed by filter before `LIMIT`), not `idx_messages_room_sent`. MAIN/BASELINE CSV latency is dominated by the full pipeline, not this query shape in isolation.

2.2.1 Core query timings (`/api/metrics` SQL shapes)

`EXPLAIN (ANALYZE, BUFFERS)` on `cs6650_chat`, rolling **24-hour** window, `user_id = '61276'` (highest-volume sender in `messages`), `room_id = '1'`.

Query	Execution	Plan / notes
Room timeline · `LIMIT 1000`	**18.15 ms**	`idx_messages_sent` + `room_id` filter
User history · `LIMIT 2000` (161 rows)	**2.95 ms**	`idx_messages_user_sent`
Active users · `COUNT(DISTINCT)` · 1 d	**~4.9–45 s**	~11.9M rows, **over 500 ms** (see diagnostics)
Rooms for user · `LIMIT 500` (20 rows)	**0.52 ms**	`idx_ura_user` (`user_room_activity`)

Active users (extra notes): The initial plan used `idx_messages_user_sent` with large heap fetches (~**4.9 s**). `idx_messages_sent (sent_at) INCLUDE (user_id)` was tested (with `idx_messages_user_sent` dropped only inside a rolled-back transaction), yielding an index-only-style scan (**Heap Fetches ~31**), but Postgres still processed ~**11.9M** `user_id` values for `COUNT(DISTINCT)` — about **42–45 s** at `work_mem = 256MB` (disk spill) and ~**45 s** at `work_mem = 2GB` (in-memory sort). The cost is dominated by sort/dedup, not only index choice. A sub-second SLA would likely need rollups, sampling, or a narrower window. The other three queries in the table behaved well on the indexes above.

2.3 Batch / flush tuning (five runs)

Five **100k** MAIN configurations with **32** workers compared batch size and flush interval; wall time, aggregate msg/s, and p50 / p95 / p99 were recorded per trial.

Trial	Batch	Flush (ms)	Wall (s)	Msg/s	p50	p95	p99
1	100	100	3.071	32,563	1,526	2,322	2,467
2	500	500	4.761	21,004	1,151	2,219	2,475
3	1,000	500	5.186	19,283	1,479	2,430	2,776
4	1,000	1,000	4.133	24,196	1,198	2,255	2,506
5	**5,000**	**1,000**	3.033	**32,971**	1,506	**2,204**	2,537

Trial **5** had the best throughput and the lowest p95, so **5000 / 1000 ms** was left in `consumer-v3 application.properties` for the full BASELINE + MAIN runs.

Terminal captures for trials **1–5** are in `load-tests/results/batch-tuning-screenshots/` (names in the first table).

Trials **2–3** were slower overall; smaller batches / shorter flush seemed to hurt sustained msg/s. **5000 / 1000 ms** means a longer possible flush wait but fewer DB round-trips, which worked better here.

---

3. System stability

3.1 Queues

During BASELINE/MAIN the `room.*` queues in RabbitMQ stayed low on **Ready**; the consumer was keeping up. A Management UI capture is saved as **`load-tests/results/fig-rabbitmq-queue-rates.png`**. After leaving the same page open and refreshing a few minutes later, **Ready** still did not pile up and the rate graph stayed in a similar band, so backlog did not look like a single short spike.

3.2 PostgreSQL

- **Connections:** ~20–24 backends to `cs6650_chat` under load (`pg_stat_*`, also `hw3-postgres-db-metrics.txt`).
- **Buffer hit ratio:** ~**90%** from `pg_stat_database` (one snapshot summed across databases gave ~89.9%).
- **Lock waits:** no rows with `wait_event_type` in `Lock` / `LWLock` in the captured samples.
- **Commits/s:** two samples of `xact_commit + xact_rollback` ~20–30 s apart gave ~**0.36–0.43** txn/s; batching explains the gap vs message rate.
- **Query throughput:** QPS was not emphasized as a headline metric because the consumer commits in **large batches**, so **`xact_commit`/s** stays low while row volume stays high. **`SELECT`** during the run was mostly metrics / checks. Per-statement QPS would need **`pg_stat_statements`** on a longer-lived instance.
- **I/O:** `pg_stat_io` shows heavy `client backend` reads/writes during the same period.

3.3 Host CPU and memory (macOS)

During **MAIN**, **Activity Monitor** → **CPU** (sorted by `% CPU`) showed **`postgres`** and the two **`java`** processes (server / consumer) as the main CPU users. Captures: **`fig-activity-monitor-cpu-main.png`** and **`fig-activity-monitor-memory-main.png`** under **`load-tests/results/`**.

3.4 Process memory

Activity Monitor → Memory for the consumer JVM during **MAIN** is **`load-tests/results/fig-activity-monitor-memory-consumer-main.png`**. Over the **12-minute** endurance-style run, RSS stayed roughly flat with no obvious slow climb.

3.5 Pools and consumer threading

- **HikariCP:** consumer `maximum-pool-size` **15**, server **20**; consumer `minimum-idle` **3**, server **5** (`application.properties` on both apps).
- **RabbitMQ:** listener **`concurrency`** / **`max-concurrency`** **20**, **`prefetch`** **50** (consumer).
- **Write-behind:** **`database-writer`** single-threaded executor drains the persistence queue; **`statistics-aggregator`** single-threaded scheduler runs minute rollups (see `PersistenceExecutorConfiguration`, `SchedulingConfiguration`).

3.6 Endurance

Endurance: **26** workers (~**81%** of 32). The run stopped with **Ctrl+C** after **12 minutes**.

`metrics-ENDURANCE-api.json`: `windowStart` `2026-03-27T20:45:00Z`, `windowEnd` `2026-03-27T20:57:00Z` (12 minutes). `metrics-ENDURANCE-room1.json` is in the same folder.

No crashes or forced restarts. `activeUsersInWindow` in that JSON is ~10^5 for the queried window. Queue depth looked stable on spot-checks of the UI during the longer runs.

Disk: Free space on the Mac was fine before and after the endurance window; Postgres **WAL/data** growth wasn't an issue for these runs.

---

4. Metrics API

After each phase the client calls `GET /api/metrics` and saves `metrics-<PHASE>-api.json`. **Client / terminal capture of the same call:** `load-tests/results/fig-metrics-api-curl.png` (day-sized `windowStart` / `windowEnd`).

maisuiyang — -zsh — 80×58

```
(base) maisuiyang@Suiyangs-MacBook-Pro ~ % curl -s "http://127.0.0.1:8081/api/metrics?windowStart=2026-03-27T00:00:00Z&windowEnd=2026-03-27T23:59:59Z&topN=10" | jq '{core, analytics: {messagesPerMinute: .analytics.messagesPerMinute[0:5], topUsers: .analytics.topUsers, topRooms: .analytics.topRooms}, windowStart, windowEnd}'
```


```
{
  "core": {
    "roomMessages": {
      "note": "roomId query parameter was not provided",
      "messages": []
    },
    "userMessageHistory": {
      "note": "userId query parameter was not provided",
      "messages": []
    },
    "activeUsersInWindow": {
      "uniqueUserCount": 100000
    },
    "roomsForUser": {
      "note": "userId query parameter was not provided for roomsForUser",
      "rooms": []
    }
  },
  "analytics": {
    "messagesPerMinute": [
      {
        "minute": "2026-03-27T21:20:00Z",
        "messageCount": 459065
      },
      {
        "minute": "2026-03-27T21:19:00Z",
        "messageCount": 32000
      },
      {
        "minute": "2026-03-27T20:47:00Z",
        "messageCount": 41571
      },
      {
        "minute": "2026-03-27T20:46:00Z",
        "messageCount": 11150
      },
      {
        "minute": "2026-03-27T20:45:00Z",
        "messageCount": 887373
      }
    ],
    "topUsers": [
      {
        "userId": "5642",
        "messageCount": 107
      },
      {
        "userId": "66647",
        "messageCount": 103
      },
      {
        "userId": "42305",
        "messageCount": 103
```

Example response shape: `core` sections such as `activeUsersInWindow`, and `analytics` with `messagesPerMinute`, `topUsers`, `topRooms`, `participationPatterns`. **`messagesPerMinute`** prefers **`analytics_minute`** (consumer rollup) and falls back to live aggregation on **`messages`** if that table has no rows in the window.

---

5. Bottlenecks

Under 1M load the slow part is mostly **persistence** (batched flush + disk) once the pipe is full. The WebSocket client still sees latency from queuing and server work, not JDBC alone.

Mitigations: **5000 / 1000 ms** batching, Hikari limits, DLQ + retry backoff, Resilience4j circuit breakers on `databaseWrites` and `roomMessages`. The metrics HTTP API is additionally guarded by **Resilience4j rate limiting** (`metricsApi`: 60 calls per second per `application.properties`) so accidental client loops cannot stampede Postgres on `/api/metrics`.

Minute-level counts for analytics come from the **`analytics_minute`** table filled by the consumer scheduler (same data a materialized view would cache; refresh is on a timer rather than a DB-native `MATERIALIZED VIEW`).

Trade-off: larger batches improve throughput but stretch worst-case time until flush.